

Práctica 2 — Programación Distribuida y Tiempo Real

Sockets: comunicaciones en red, Vagrant y experimentación de tiempos

Asistente de IA utilizado: Claude (Anthropic) — Antigravity IDE.
Se deja constancia de su uso en los puntos 1 y 4 donde el enunciado lo requiere explícitamente.

Ejercicio 1 — Ubuntu LTS más reciente en Vagrant

Versión seleccionada

Aspecto	Valor
Box de Vagrant	ubuntu/noble64
Versión de Ubuntu	24.04 LTS “Noble Numbat”
Origen	Canonical (publicado en Vagrant Cloud)
Kernel	Linux 6.8+
Soporte LTS	Hasta abril de 2029 (5 años)

Comparación con la versión usada en el TP1

En el TP1 se usó `ubuntu/jammy64` (Ubuntu 22.04 LTS “Jammy Jellyfish”). Las principales diferencias son:

Aspecto	Ubuntu 22.04 (Jammy)	Ubuntu 24.04 (Noble)
Kernel	5.15	6.8

GCC	11.x	13.x / 14.x
Python	3.10	3.12
Systemd	249	255+
OpenSSL	3.0	3.2+
Soporte hasta	Abril 2027	Abril 2029

Consulta a la IA: ¿qué versión conviene más?

Pregunta realizada a Claude (Anthropic):

“¿Para experimentación con sockets TCP en C dentro de Vagrant, es mejor usar Ubuntu 22.04 LTS o Ubuntu 24.04 LTS? ¿Por qué?”

Respuesta resumida de la IA:

Para experimentación académica con sockets TCP en C, **ambas versiones son perfectamente válidas**. Sin embargo, Ubuntu 24.04 LTS tiene algunas ventajas marginales:

- **Kernel 6.8** incluye mejoras en el stack de red TCP (TCP-AO, mejoras en BBRv3) que pueden afectar sutilmente las mediciones de latencia en escenarios de alta carga.
- **GCC 13/14** genera código ligeramente más optimizado para operaciones de memoria (relevante para copias de buffers grandes).
- **Soporte más largo** (2 años adicionales de actualizaciones de seguridad).

La IA también señaló que Ubuntu 22.04 puede ser preferible si se busca **máxima estabilidad** y compatibilidad con software legacy, ya que tiene más tiempo en producción y bugs resueltos. Para nuestro caso de uso (experimentos simples de sockets en una red local/virtual), la diferencia práctica es mínima.

Comentario propio: La respuesta de la IA es razonable. Para esta práctica, la elección de 24.04 se justifica simplemente porque el enunciado pide usar una **versión más reciente** que la del TP1.

Vagrantfile utilizado

```
# Vagrantfile.punto1
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/noble64"
  config.vm.hostname = "noble-tp2"
```

```
config.vm.provider "virtualbox" do |vb|  
  vb.memory = "512"  
  vb.name = "TP2-Ubuntu-Noble-24.04"  
end  
config.vm.provision "shell", inline: <<-SHELL  
  apt-get update  
  apt-get install -y net-tools joe zip build-essential  
SHELL  
end
```

Cómo ejecutarlo

```
# Copiar el Vagrantfile  
cp Vagrantfile.punto1 Vagrantfile  
  
# Levantar la VM (descarga la imagen la primera vez)  
vagrant up  
  
# Conectarse por SSH  
vagrant ssh  
  
# Dentro de la VM, verificar la versión:  
lsb_release -a  
uname -r
```

Capturas de pantalla

TODO: Insertar aquí dos capturas:

1. La máquina virtual "TP2-Ubuntu-Noble-24.04" visible en la GUI de VirtualBox.
2. La terminal SSH mostrando la salida de `lsb_release -a` y `uname -r`.

Ejercicio 2 — Experimentos entre dos computadoras físicas

*Nota: Este ejercicio requiere dos computadoras físicas conectadas en red.
Se utilizan los mismos programas `server_timed` y `client_timed` del TP1.*

Descripción del entorno

	Computadora A (Servidor)	Computadora B (Cliente)
CPU	TODO: completar	TODO: completar
RAM	TODO: completar	TODO: completar
SO	TODO: completar	TODO: completar
Red	TODO: misma LAN / redes distintas	TODO: misma LAN / redes distintas

Cómo ejecutar los experimentos

En la Computadora A (servidor):

```
gcc -O2 -o server_timed server_timed.c
./server_timed 5555 srv_fisico.csv
```

En la Computadora B (cliente):

```
gcc -O2 -o client_timed client_timed.c
for n in 10 100 1000 10000 100000 1000000; do
    ./client_timed <IP_COMPUTADORA_A> 5555 $n cli_fisico.csv
done
```

Resultados

TODO: Completar con los datos reales de la corrida.

Generar gráficos con: `python3 plot_times_tp2.py cli_fisico.csv srv_fisico.csv`
`pingpong_fisico.csv`

Análisis (Punto 2c)

Los resultados esperables son los mismos del análisis del TP1, con estas diferencias al ejecutar en red física:

- **write()** : Para tamaños chicos (10-1000 bytes), el tiempo está dominado por el overhead del syscall, **no es proporcional** al tamaño. Para tamaños grandes (100K-1M bytes), el tiempo crece significativamente porque el buffer de envío del socket

del kernel se llena y `write()` debe bloquearse esperando que TCP envíe y reciba ACKs del otro extremo para liberar espacio.

- **`read()`** : El tiempo por llamada individual (buffer fijo de 255 bytes) se mantiene **aproximadamente constante**, ya que lo que cambia es la cantidad de llamadas, no el costo de cada una. En red física, el piso de tiempo por `read()` será más alto que en loopback por la latencia de red.
 - **Proporcionalidad**: No es válido asumir que 1000 bytes tarda 10× más que 100 bytes. La relación no es lineal por el costo fijo del syscall y el comportamiento de los buffers TCP.
-

Ejercicio 3 — Experimento Ping-Pong (RTT / 2)

Concepto

El experimento “ping-pong” consiste en:

1. El cliente envía N bytes al servidor.
2. El servidor recibe los N bytes completos y los **reenvía de vuelta** al cliente.
3. El cliente mide el **tiempo total de ida y vuelta (RTT)**.
4. Se estima el tiempo en una dirección como **RTT / 2**.

Código desarrollado

Se crearon dos programas nuevos:

`server_pingpong.c`

```
// Protocolo:  
// 1. Lee 4 bytes con el total de bytes que va a recibir.  
// 2. Recibe los N bytes completos en un buffer dinámico.  
// 3. Reenvía los N bytes completos de vuelta (eco total).
```

El servidor asigna un buffer dinámico (`malloc(total)`) del tamaño exacto que le anuncia el cliente, lee todos los bytes, y luego los reescribe completos con un bucle de `write()` que maneja escrituras parciales.

¿Por qué un buffer dinámico? A diferencia del `server_timed` que lee en chunks de 255 bytes (para medir cada `read()` individual), acá necesitamos acumular todos los datos recibidos para poder reenviarlos. El tamaño puede ser hasta 1MB, así que se usa `malloc()` .

client_pingpong.c

```
// Medición:
// t0 = clock_gettime(CLOCK_MONOTONIC)
// write(N bytes)    -- envío completo
// read(N bytes)     -- recepción del eco completo
// t1 = clock_gettime(CLOCK_MONOTONIC)
// RTT = t1 - t0
// T_unidireccional ≈ RTT / 2
```

El cliente mide el RTT **desde antes del primer write() hasta después de recibir el último byte de vuelta**. Esto captura el tiempo total de ida (write + transmisión + recepción en servidor + procesamiento) y vuelta (reenvío + transmisión + recepción en cliente).

Cómo ejecutarlo

En la computadora servidor:

```
gcc -O2 -o server_pingpong server_pingpong.c
./server_pingpong 5556
```

En la computadora cliente:

```
gcc -O2 -o client_pingpong client_pingpong.c
for n in 10 100 1000 10000 100000 1000000; do
    ./client_pingpong <IP_SERVIDOR> 5556 $n pingpong.csv
done
```

Comparación Punto 2 vs Punto 3

La diferencia clave entre los dos métodos es:

Aspecto	Punto 2 (write() directo)	Punto 3 (RTT/2)
Qué mide	Solo el tiempo de la syscall write() local	Tiempo de ida + vuelta completo
Incluye red	Parcialmente (si el buffer se llena)	Siempre

Supuesto	Ninguno	Simetría del canal (ida = vuelta)
Valor esperado	$RTT/2 \geq \text{write}()$ siempre	Mayor que $\text{write}()$ solo

RTT/2 siempre será $\geq$ al tiempo de `write()` del Punto 2, porque RTT/2 incluye:

- El tiempo de transmisión por la red (que `write()` no mide si cabe en el buffer).
- El tiempo de procesamiento del servidor.
- El tiempo de transmisión de vuelta.

La estimación RTT/2 **asume simetría del canal**: que el tiempo de ida es igual al de vuelta. En redes reales (especialmente WiFi o conexiones asimétricas como ADSL), esta suposición puede no ser exacta.

Resultados

TODO: Completar con los datos reales. Generar el gráfico comparativo con:

```
python3 plot_times_tp2.py cli_fisico.csv srv_fisico.csv pingpong.csv
```

Ejercicio 4 — Scripts de despliegue con Vagrant

El problema del asincronismo

Cuando se automatizan los experimentos con Vagrant, ambas VMs se aprovisionan secuencialmente (o en paralelo). El problema es: **¿qué pasa si el proceso cliente intenta conectarse antes de que el servidor haya hecho `bind()` + `listen()` ?**

El `connect()` del cliente falla con `ECONNREFUSED` ("Connection refused") porque el kernel del servidor no tiene un socket pasivo en ese puerto y responde con un paquete `RST` al `SYN` del cliente.

Solución implementada: doble mecanismo de sincronización

Se resolvió con **dos capas de verificación** en `run_client.sh` :

Capa 1: Señalización por archivo compartido

```
# El servidor crea este archivo cuando está listo:
```

```
echo "ready" > /vagrant/.server_ready
```

```
# El cliente espera a que exista:
```

```
while [ ! -f /vagrant/.server_ready ]; do
```

```
    sleep 2
```

```
done
```

Funciona porque `/vagrant` es una carpeta compartida entre ambas VMs (y el host). Cuando el servidor escribe el archivo, el cliente lo ve inmediatamente a través del filesystem compartido de VirtualBox.

Capa 2: Verificación de puerto TCP con reintentos

```
# Después de que el archivo exista, verificar que el puerto esté realmente escuchando:
```

```
while ! (echo > /dev/tcp/$host/$port) 2>/dev/null; do
```

```
    sleep 2
```

```
done
```

Esta segunda capa es necesaria porque el archivo podría crearse *antes* de que el proceso `server_timed` realmente haya ejecutado `listen()`. La verificación con `/dev/tcp` intenta abrir una conexión TCP real al puerto y solo continúa cuando tiene éxito.

Escenario 4a: Dos máquinas virtuales (VM a VM)

red privada		
vm_server	192.168.56.10/11	vm_client
server_timed:5555		client_timed
server_pp:5556		client_pingpong
/vagrant (compartida)		

Vagrantfile.punto4a levanta dos VMs con IPs fijas en una `private_network` :

- `vm_server` (192.168.56.10): compila y lanza los servidores.
- `vm_client` (192.168.56.11): compila los clientes, espera al servidor, y ejecuta todos los experimentos automáticamente.

Los resultados quedan en `/vagrant/results/` (= la carpeta local `results/`).

Cómo ejecutarlo:

```
cp Vagrantfile.punto4a Vagrantfile
vagrant up
# Los experimentos se ejecutan automáticamente durante el provisioning.
# Los resultados quedan en results/
ls results/
```

Escenario 4b: Una VM + el host

		forwarded_port	
VM (servidor)	5555	5555	HOST
	5556	5556	(cliente)
server_timed:5555			client_timed
server_pp:5556			client_pp

Vagrantfile.punto4b levanta una sola VM con servidores en los puertos 5555 y 5556, redirigidos al host mediante `forwarded_port`. El cliente se ejecuta desde el host contra `127.0.0.1`.

Cómo ejecutarlo:

```
cp Vagrantfile.punto4b Vagrantfile
vagrant up
# Los servidores ya están corriendo en la VM.
# Desde el host (necesita gcc / WSL / o similar):
bash run_client_host.sh
```

Consulta a la IA: mejor método para estimar tiempos de comunicación

Pregunta realizada a Claude (Anthropic):

"¿Cuál es el mejor método experimental para estimar el tiempo de comunicación entre dos computadoras diferentes?"

Respuesta resumida de la IA:

El método más robusto es el de **ping-pong (RTT/2)** con las siguientes mejoras:

1. **Múltiples iteraciones:** Repetir cada medición al menos 100 veces y calcular la mediana (no el promedio, para evitar outliers por jitter de red o scheduling del SO).
2. **Descarte de warm-up:** Las primeras mediciones suelen tener latencia elevada por resolución ARP, establecimiento de conexión TCP, y cachés frías. Descartarlas.
3. **Sincronización de relojes:** Si se quiere medir unidireccionalmente, usar NTP o PTP para sincronizar los relojes de ambas máquinas. Sin embargo, NTP tiene una precisión típica de $\pm 1-10\text{ms}$, lo cual puede ser mayor que la propia latencia a medir en una LAN.
4. **Control del entorno:** Deshabilitar power saving (CPU frequency scaling), fijar la frecuencia del CPU, y minimizar tráfico de red concurrente.

Comentario propio / Crítica de la respuesta:

La respuesta de la IA es mayormente correcta, pero tiene un problema potencial: **la suposición de simetría de RTT/2 no siempre es válida**. En redes WiFi, la latencia de subida puede ser significativamente diferente a la de bajada. Además, la IA no menciona que en entornos virtualizados (como nuestro caso con Vagrant/VirtualBox), la virtualización del adaptador de red agrega una capa adicional de variabilidad que no existe en hardware real, lo que puede hacer que los resultados sean menos reproducibles.

Otro punto que la IA no enfatiza suficientemente es el **efecto del tamaño del mensaje en la simetría**: para mensajes muy grandes (1MB), el tiempo de transmisión domina y la simetría se mantiene mejor; para mensajes muy chicos (10 bytes), el overhead del protocolo y el jitter son proporcionalmente mayores, haciendo la estimación RTT/2 menos precisa.

Ejercicio 5 — Impacto del orden de ejecución y el desfase temporal

Pregunta del enunciado

¿Los resultados de tiempo se ven afectados por el orden de ejecución y las diferencias de tiempo iniciales entre los programas? ¿Qué sucede si hay una diferencia de 10 segundos en el inicio?

Experimento diseñado

Se creó `client_sleep_test.c` con 4 modos de operación, cada uno insertando un `sleep(10)` en un punto diferente del ciclo de vida de la conexión:

Modo	Dónde se inserta el sleep(10)	Efecto esperado
0	Sin sleep (baseline)	Referencia para comparar
1	ANTES de connect()	El servidor ya hizo listen() . El cliente simplemente tarda 10s más en conectar. No afecta los tiempos de write/read una vez conectado.
2	DESPUÉS de connect() , ANTES de write()	La conexión TCP ya está establecida (3-way handshake completado). El servidor está bloqueado en read() esperando datos. No afecta los tiempos de transferencia.
3	DESPUÉS de write() , ANTES de read()	El cliente envió todos los datos. El servidor recibe, procesa y reenvía los datos. Los datos de vuelta llegan al buffer de recepción del kernel del cliente y quedan ahí esperando durante 10 segundos. Cuando el read() finalmente se ejecuta, los datos ya están en el buffer local el read() será más rápido de lo normal (casi instantáneo, sin esperar la red).

Explicación técnica

¿Por qué el sleep antes de connect/write no afecta las mediciones?

Porque la medición de tiempos se hace **alrededor de cada operación individual**, no del programa completo:

```
// El sleep ocurre FUERA de la zona medida:
sleep(10);           // no se mide esto
clock_gettime(CLOCK_MONOTONIC, &t0); // inicio de medición
write(sockfd, data, total);
clock_gettime(CLOCK_MONOTONIC, &t1); // fin de medición
```

El sleep() solo retrasa el momento en que se ejecuta la operación, pero no cambia la velocidad de la red ni el tamaño de los buffers.

¿Qué pasa con el sleep DESPUÉS del write (Modo 3)?

Este es el caso más interesante. Lo que sucede durante esos 10 segundos:

1. El cliente hizo `write()` de N bytes los datos viajan al servidor por TCP.
2. El servidor recibe los N bytes completos y hace `write()` de vuelta.
3. Los datos de vuelta viajan del servidor al cliente **llegan al buffer de recepción del kernel del cliente** (`SO_RCVBUF`).
4. El cliente está durmiendo (`sleep(10)`) nadie llama a `read()` .
5. Los datos se acumulan en el buffer del kernel. Si $N < \text{tamaño del buffer}$ (típicamente 128K-256K), todo cabe sin problema.
6. Cuando el `sleep(10)` termina y el cliente llama a `read()` , los datos **ya están en memoria local** la lectura es casi instantánea (solo copia de kernel a espacio de usuario, sin esperar la red).

Resultado: El `read()` del Modo 3 será significativamente **más rápido** que el baseline, porque no incluye el tiempo de red (ya llegaron los datos).

¿Y si el buffer se llena? (caso de 1MB)

Si los N bytes de vuelta son mayores que el buffer de recepción, TCP usa su mecanismo de **control de flujo por ventana (TCP Window Advertisement)**:

- El buffer del receptor se llena.
- El receptor anuncia una ventana de 0 bytes.
- El emisor (servidor) se bloquea en su `write()` esperando que el receptor libere espacio.
- Cuando el cliente despierta y hace `read()` , libera espacio en el buffer el servidor puede seguir enviando.

En este caso, el `sleep(10)` del Modo 3 **sí afecta** el rendimiento global porque fuerza al servidor a pausar su transmisión.

Cómo ejecutar el experimento

```
# En una terminal: servidor pingpong
./server_pingpong 5556

# En otra terminal: experimento completo
bash run_sleep_experiment.sh 127.0.0.1 5556
# Los resultados quedan en results/sleep_experiment.csv
```

Conclusión del Punto 5

Situación	¿Afecta los tiempos de transferencia?
Servidor arranca 10s antes que el cliente	No. El servidor espera en <code>accept()</code> hasta que el cliente conecte.
Cliente arranca 10s antes que el servidor	Sí <code>connect()</code> falla con <code>ECONNREFUSED</code> . Por eso se necesita la solución de asincronismo del Punto 4.
Sleep antes de connect (modo 1)	No afecta write/read.
Sleep antes de write (modo 2)	No afecta write/read.
Sleep después de write, antes de read (modo 3)	Sí afecta read: lo hace artificialmente más rápido porque los datos ya están en el buffer local. Para datos > tamaño del buffer, también afecta al emisor (control de flujo TCP).

Archivos entregados

Código fuente

Archivo	Descripción	Punto
<code>server_timed.c</code>	Servidor que mide tiempos de cada <code>read()</code>	2
<code>client_timed.c</code>	Cliente que mide tiempos de cada <code>write()</code>	2
<code>server_pingpong.c</code>	Servidor eco (recibe N bytes, reenvía N bytes)	3
<code>client_pingpong.c</code>	Cliente que mide RTT (write + read ida y vuelta)	3
<code>client_sleep_test.c</code>	Cliente con sleep configurable en 4 modos	5

Scripts de automatización

Archivo	Descripción	Punto
<code>run_server.sh</code>	Lanza ambos servidores en background	4

run_client.sh	Cliente con espera activa + todos los experimentos	4
run_client_host.sh	Cliente para escenario VM Host	4b
run_sleep_experiment.sh	Automatiza los 4 modos de sleep	5

Configuración Vagrant

Archivo	Descripción	Punto
Vagrantfile.punto1	Ubuntu 24.04 LTS (Noble) - verificación de versión	1
Vagrantfile.punto4a	2 VMs (VM a VM) con red privada	4a
Vagrantfile.punto4b	1 VM (servidor) + host (cliente) con forwarded_port	4b

Graficación

Archivo	Descripción
plot_times_tp2.py	Genera los 4 gráficos del TP2 a partir de los CSVs